



Treinamento SQL - Essentials

LUIS_DEV



Funções básicas SQL

O que são funções?

As funções básicas são procedimentos nativos do banco de dados na qual facilitam certas operações que se deseja fazer dentro de uma query para gerar um determinado resultado. O SQL Server tem os seguintes grupos de funções: Funções de cadeia de caracteres, conversão, matemática, agregação, data e hora. Contudo já vimos sobre as funções de agregação na última aula.

Funções matemáticas

São um conjunto de funções que fazem cálculos baseados em valores de entrada fornecidos como parâmetros às funções e retornam valores numéricos. Dentro deste grupo de funções temos por exemplo, exponencial, logarítmica, arredondamento e etc. Abaixo vou deixar o link com maior detalhamento sobre as funções.



Funções básicas SQL

Funções matemáticas

Exemplo

```
select id_instrutor, round(salario,1) salario_round, salario from graduacao.instrutor;
```

```
select id_instrutor, round(salario,0) salario_round, salario from graduacao.instrutor;
```

Link do material de aprofundamento

[Funções matemáticas](#)

LUIS_DEV



Funções básicas SQL

Funções de conversão

São um conjunto de funções que oferecem suporte à conversão de tipo de dados, dentro deste grupo temos como principais funções o CAST, CONVERT e PARSE. As funções CAST e CONVERT possuem o mesmo objetivo, contudo a diferença entre elas é a qual padrão respeitado para o desenvolvimento o CAST é padrão ISO e o CONVERT é o padrão T-SQL. Contudo sua finalidade é a conversão de tipo de dados e sua delimitação. Em relação ao PARSE, é um função que faz a conversão baseando-se na localidade que a sessão o SQL Server está executando ou através da definição da região na construção da query e os tipos de dados que o PARSE suporta são: data/hora e tipos numéricos. Para as demais conversões utilizar os modelos anteriores.

Exemplo:

```
select id_instrutor, cast(salario as int) salario_int, salario from graduacao.instrutor;
```

```
select nome_departamento, convert(decimal(7,1), orcamento,2) salario_decimal, orcamento from graduacao.departamento;
```

Link do material de aprofundamento

[Funções conversão](#)



Funções básicas SQL

Funções de cadeia de caracteres

São um conjunto de funções que oferecem operação com cadeias de caracteres que podem ir concatenação de strings à desmantelamento de textos. Algumas funções deste grupo são: LTRIM, RTRIM, TRIM, RIGHT, LEFT, UPPER, SUBSTRING, ETC..

Exemplo:

```
select id_instrutor, upper(nome) nome_upper, salario from graduacao.instrutor;
```

```
select id_instrutor, nome, salario from graduacao.instrutor where upper(nome) = 'ARIAS';
```

```
select id_aluno, nome, substring(nome,1,3) nome_substring_1, substring(nome,4,3) nome_substring_2 from graduacao.aluno;
```

```
select id_aluno, nome, replace(substring(nome,1,3),'r','m') from graduacao.aluno;
```

```
select id_aluno, nome, replace(id_aluno,'1','00001') from graduacao.aluno;
```

Link do material de aprofundamento

[Funções cadeia de caracteres](#)



Funções básicas SQL

Funções de data e hora

São um conjunto de funções a possibilidade de trabalhar com data e hora, isso se estende de formatação de datas, cálculo entre datas, mudança de fuso horário, mudança da região geográfica, etc.. Há N possibilidades de como utilizar as funções de datas, podem ser utilizadas nas cláusulas SELECT, WHERE, JOIN, UPDATE, INSERT.

Link do material de aprofundamento

[Funções datas e hora](#)

LUIS_DEV



Funções básicas SQL

Exemplo

```
select id_aluno, nome, sysdatetime() data_sistemica from graduacao.aluno;
```

```
select id_aluno, nome, current_timestamp current_date_1 from graduacao.aluno;
```

```
select id_aluno, nome, month(current_timestamp) mes from graduacao.aluno;
```

```
select id_aluno, nome, day(current_timestamp) dia from graduacao.aluno;
```

```
select id_aluno, nome, year(current_timestamp) ano from graduacao.aluno;
```

```
select id_aluno, nome, datediff(year, format(current_timestamp, 'yyyy-MM-dd'), '2020-01-01') diff_date from graduacao.aluno;
```

```
select id_aluno, nome, format(current_timestamp, 'yyyy-MM-dd') format_date from graduacao.aluno;
```



Funções T-SQL

Funções de usuários

As funções que vamos ver agora, são funções desenvolvidas a partir de comandos semelhantes aos das linguagens de programação mais comuns, podendo usar estruturas condicionais e de repetição, por exemplo. As vantagens de ter essas funções são as possibilidades de estender o seu uso para outros usuários, a capacidade de personalização dos resultados a partir do código desenvolvido dentro da função. Ao definir uma função é possível gerar retornos a partir dos tipos de dados declarados na construção da mesma. As funções podem retornar resultados escalares e criar uma tabela como resultado. Agora vamos ver como construir uma função, elementos necessários e aplicação.

Link do material de aprofundamento

[Material Funções usuários](#)

LUIS_DEV



Funções T-SQL

Elementos de uma função

Funções escalares

```
CREATE FUNCTION NOME_DA_FUNCTION  
(  
  @PARAMETRO TIPO_PARAMETRO  
)  
RETURNS INT  
AS  
BEGIN  
  DECLARE @PARAMETRO TIPO_PARAMETRO  
  
  SET @PARAMETRO = CALCULO  
  
  RETURN @PARAMETRO  
END  
GO
```

Funções de tabela

```
CREATE FUNCTION NOME_DA_FUNCTION  
(@PARAMETRO TIPO_PARAMETRO)  
RETURNS TABLE  
AS  
RETURN  
(  
  SELECT  
    CAMPO1,CAMPO2,CAMPO3,CAMPO4  
  FROM TABELA  
  WHERE CAMPO1 = @PARAMETRO  
)  
GO
```



Funções T-SQL

Elementos de uma função

Create function - Essa instrução tem como objetivo iniciar o processo de criação da função do usuário, nesta instrução temos as seguintes passos, definir o schema de banco de dados no qual a função vá pertencer, o próximo passo é definir um nome exclusivo para a função, definir entre parênteses os parâmetros que a função vai receber, começando por @nome_param e escolhendo o tipo de dados do parâmetro.

Return - O retorno pode ser escalar, gerando um resultado a partir da variável na qual declaramos dentro da função ou retorno de tabela, no qual o resultado do retorno é uma tabela, pode se definir o schema dessa tabela ou gerar o resultado a partir de um SELECT. Importante lembrar que para os casos de definição de schema, definir o nome da tabela com @nome_tabela.

Execute as with - Comando utilizado no sentido definir o tipo de chamada da função, criando o modo no qual a função pode ser chamada dentro das consultas pelos os usuários. Esse comando é específico para funções escalares.

Begin - O comando para referenciar o início do bloco de escrita da função.

Declare - Comando para declaração das variáveis globais da função, que podem também ser retorno da função.



Funções T-SQL

Elementos de uma função

END - Comando para finalizar o bloco de instrução da função.

Exemplo

```
create function graduacao.qntd_aumenta_salario(@range_salario numeric(10,2))  
  
    returns int  
    with execute as CALLER  
    as  
    begin  
  
        declare @quantidade_aumentos int;  
  
        set @quantidade_aumentos = (select count(*) from graduacao.instrutor  
        where salario <= @range_salario)  
  
        return @quantidade_aumentos;  
  
    end;  
  
select graduacao.qntd_aumenta_salario(60000.00);
```



Funções T-SQL

Exemplo

```
create function graduacao.semestre_salario(@semestre
varchar(15),@ano numeric(4,0))

returns table
as
return
(

select a.id_professor, b.salario, b.salario * 0.02 salario_adiciona,
b.nome, c.titulo from graduacao.professor a
inner join graduacao.instrutor b on a.id_professor = b.id_instrutor
inner join graduacao.curso c on a.id_curso = c.id_curso
where semestre = @semestre and ano = @ano);

select * from graduacao.semestre_salario('primavera',2004);

drop function graduacao.semestre_salario;
```



PROCEDURE T-SQL

O que é Procedure?

A PROCEDURE é um procedimento armazenado com um conjunto de comandos SQL que executam atividades específicas dentro do bloco de desenvolvimento. A forma de comportamento de uma PROCEDURE é parecida com as funções, podendo gerar resultados de várias saídas como comportamento de retorno. Outro ponto, é possível executar vários comandos. Os procedimentos armazenados também aceitam parâmetros de entrada para que possam ser usados de forma diferente dependendo da preferência do usuário. A PROCEDURE traz alguns benefícios ao serem utilizadas. Como redução no tráfego na rede, pode ser triggada, ajuda na estabilidade de performance de um banco de dados, diminui riscos, criar rotinas de processamento, etc.. Além disso, o PROCEDURE permite alterações dentro do banco de dados onde está depositada, por exemplo, podemos utilizar comandos de update, insert, delete, etc. O procedimento é uma ferramenta importante para os DBAs, pois os mesmos podem gerar uma rotina de execução de scripts encapsulados dentro do bloco de código de uma PROCEDURE.

Link do material de aprofundamento

[Material PROCEDURE](#)

LUIS_DEV



PROCEDURE T-SQL

Benefícios das procedures

Pode ser facilmente modificado - Podemos modificar facilmente o código dentro do procedimento armazenado sem a necessidade de reiniciar ou implantar o aplicativo. Por exemplo, se as consultas T-SQL forem escritas no aplicativo e precisarmos alterar a lógica, devemos alterar o código no aplicativo e implantá-lo novamente. Os procedimentos armazenados do SQL Server eliminam esses desafios armazenando o código no banco de dados, então, quando queremos mudar a lógica dentro do procedimento, podemos simplesmente fazê-lo pela simples instrução ALTER PROCEDURE ou DROP PROCEDURE.

Tráfego de rede reduzido - Armazenar consultas T-SQL em procedimentos em vez de código de aplicativo reduz o tráfego de rede. Isso ocorre porque apenas os nomes dos procedimentos são enviados pela rede em vez de todo o código T-SQL.

Reutilizável - Os procedimentos armazenados podem ser executados por vários usuários ou vários aplicativos clientes sem a necessidade de escrever o código novamente.

Segurança - Os procedimentos armazenados reduzem a ameaça eliminando o acesso direto às tabelas. também podemos criptografar os procedimentos armazenados ao criá-los para que o código-fonte dentro do procedimento armazenado não fique visível.

Desempenho - O procedimento armazenado do SQL Server, quando executado pela primeira vez, cria um plano e o armazena no buffer pool para que o plano possa ser reutilizado na próxima execução.



PROCEDURE T-SQL

Tipos de procedures

Locais - As PROCEDURES criadas dentro do schema do banco de dados do usuários, são usadas dentro do escopo do banco e do permissionamento;

Temporários - As PROCEDURES temporárias são de dois tipos, locais que estão limitadas ao escopo do schema do banco, Para criar um procedimento temporário local o nome deve começar com #. As temporárias globais, que podem ser utilizadas no escopo geral dos schemas de um banco de dados.. Para criar um procedimento temporário global o nome deve começar com ##

Procedimentos de Sistema - As PROCEDURES sistêmicas ficam armazenadas no banco de dados master, para identificar esse tipo de PROCEDURE, basta buscar pelas que usam as iniciais sp derivada de procedimentos armazenados. O objetivo deste tipo de procedimento é executar tarefas administrativas que podem afetar a todos os bancos, este tipo de ação usualmente é utilizada pelos DBAs.

Remotos - Podemos usar Queries Distribuídas para tais procedures. São utilizadas apenas para compatibilidade.

Estendidos - Diferente dos procedimentos já citados, este tipo de procedimento recebe a extensão .dll e são executadas fora do SGBD SQL Server. São identificadas com o prefixo xp.



PROCEDURE T-SQL

Construção básica de uma procedure

```
CREATE [ OR ALTER ] [ PROC | PROCEDURE ] [schema_name.]  
procedure_name (@parâmetro_1 tipo,  
@parâmetro_2 tipo output)  
AS  
BEGIN
```

Código que deseja executar

```
END
```

LUIS_DEV



PROCEDURE T-SQL

Elementos de uma procedure:

Create Procedure - Essa instrução tem como objetivo iniciar o processo de criação da procedure, nesta instrução temos as seguintes passos, definir o schema de banco de dados no qual a procedure vá pertencer, o próximo passo é definir um nome exclusivo para a procedure, definir entre parênteses os parâmetros que a função vai receber, começando por @nome_param e escolhendo o tipo de dados do parâmetro e por fim determinar se é um parâmetro de saída OUT/OUTPUT ou de entrada IN.

Return - O retorno pode ser escalar, gerando um resultado a partir da variável na qual declaramos dentro da função ou retorno de tabela, no qual o resultado do retorno é uma tabela, pode se definir o schema dessa tabela ou gerar o resultado a partir de um SELECT. Importante lembrar que para os casos de definição de schema, definir o nome da tabela com @nome_tabela.

Execute as with - Comando utilizado no sentido definir o tipo de chamada da função, criando o modo no qual a função pode ser chamada dentro das consultas pelos os usuários.

Begin - O comando para referenciar o início do bloco de escrita da procedure.

Declare - Comando para declaração das variáveis globais da procedure, que podem também ser retorno da procedure.



PROCEDURE T-SQL

Como chamar uma PROCEDURE:

declare @parameter_out tipo dados;

exec nome_da_procedure parameter_1,@parameter_out ;

A cláusula declare só é necessária em casos nos quais as procedures são criadas referenciando parâmetros como output, que tem como objetivo de gerar um retorno escalar do cálculo da procedure.

Exemplo

```
create procedure graduacao.proc_count_dept(@nome_dept varchar(20), @count_d integer out)
as
begin
    select count(*) count_d from graduacao.instrutor where
    instrutor.nome_departamento = @nome_dept
end
declare @count_d int;
exec graduacao.proc_count_dept 'Physics',@count_d ;
```



PROCEDURE T-SQL

Exemplo:

```
create procedure graduacao.relatorio_aluno_semestre(@semestre varchar(20), @ano integer)
as
begin
    if @semestre = 'primavera'
        select * from graduacao.nota_aluno where nota_aluno.semestre = @semestre
        and nota_aluno.ano = @ano
    else
        select * from graduacao.nota_aluno where nota_aluno.semestre
        <> @semestre and nota_aluno.ano = @ano
end

exec graduacao.relatorio_aluno_semestre 'primavera', 2002 ;
```



PROCEDURE T-SQL

Exemplo:

```
create procedure graduacao.#aumento_salario(@range_salario numeric(8,2), @aumento
decimal(4,3))
as

begin

    update graduacao.instrutor
    set salario = salario * @aumento
    where salario <= @range_salario
    print 'Aumento salarial aplicado'

end

drop procedure graduacao.#aumento_salario;

exec graduacao.#aumento_salario 60000.00, 1.2;
```



CREATE Tabelas temporárias

O que são tabelas temporárias?

É um tipo de tabela que é de possível utilização enquanto a sua conexão com o banco estiver ativa, quer dizer que a existência da tabela está ligado a sessão do usuário, quando a sessão é encerrada a tabela temporária desaparece. As tabelas temporárias ficam no tempdb, fora do ambiente produtivo. As tabelas temporárias tem algumas vantagens na utilização, como por exemplo: Fazer testes antes de subir a tabela para produção, alterar dados de uma tabela produtiva a partir de uma temporárias antes de aplicar as mudanças na original e evitar gerar tabelas desnecessárias no ambiente produtivo.

Link do material de aprofundamento

[Material tabelas temporárias](#)

LUIS_DEV



CREATE Tabelas temporárias

Tipos de tabelas temporárias:

Tabelas temporárias locais - São tabelas que estão dentro do escopo de sessão do usuário, assim que usuário terminar a sessão a tabela será destruída. Essa tabela pode ser utilizada dentro das transações dentro da sessão do usuário.

Tabelas temporárias globais - As temporárias globais ficam visíveis para qualquer sessão, com isso qualquer sessão pode acessar ou alterar essa tabela. A vantagem é que a tabela, em caso de testes ou cruzamento de análises, gera a possibilidade de trabalho simultâneo, mas o que é uma vantagem pode ser uma desvantagem, pois qualquer usuário pode alterar a estrutura da tabela ou deletar dados. A tabela será excluída a partir do momento que o criador da mesma encerrar sua sessão. Essa tabela pode ser utilizada dentro das transações dentro das sessões dos usuários.

Tabelas variáveis - São tabelas geradas a partir da cláusula DECLARE + TABLE, o funcionamento é bem parecido com o modelo da tabela temporária local, funciona em um escopo do batch do script SQL que você estiver escrevendo, em suma, essa tabela não fica disponível para outras queries da mesma sessão. As tabelas variáveis podem ser utilizadas com INSERT, SELECT, UPDATE, DELETE. Um ponto importante é que sempre que for necessário executar o DECLARE quando for utilizar a tabela.



CREATE Tabelas temporárias

Exemplo:

Locais

```
create table #departamento
(nome_departamento varchar(50),
predio                varchar(30),
orcamento             numeric(12,2) check (orcamento >
o),
primary key (nome_departamento)
);
```

Globais

```
create table ##sala_aula
(predio                varchar(15),
numero_sala           varchar(10),
capacity              numeric(3),
primary key (predio, numero_sala)
);
```

Variáveis

```
declare @grade_horario table
(id_horario           varchar(4),
dia                   varchar(1),
inicio_hora           numeric(2) check
(inicio_hora >= 0 and inicio_hora < 24),
inicio_min             numeric(2)
check (inicio_min >= 0 and inicio_min < 60),
fim_hora               numeric(2)
check (fim_hora >= 0 and fim_hora < 24),
fim_min                numeric(2)
check (fim_min >= 0 and fim_min < 60),
primary key (id_horario, dia,
inicio_hora, inicio_min)
);
```



Views

O que são Views?

É uma tabela virtual gerada a partir de uma query, a VIEW permite que a query fique encapsulada e que seus resultado seja gerado em tempo de execução, o que isso quer dizer? Todo resultado vindo da VIEW é gerado virtualmente, não há dado físico no banco, logo os dados são temporários enquanto a VIEW estiver em execução. As vantagens da VIEWS são: O controle da segurança do usuário para acessar os dados das tabelas do banco de dados. Podemos permitir que os usuários obtenham os dados da VIEW, e o usuário não precisa de permissão para cada tabela ou coluna buscar dados. Além disso podemos aplicar funções dentro da VIEW que podem criptografar informações de determinadas colunas das tabelas.



Views

Exemplo

```
create view graduacao.relatorio_professores
```

```
as
```

```
select a.id_professor, b.salario, b.salario * 0.02 salario_adiciona, b.nome, c.titulo from graduacao.professor a
```

```
inner join graduacao.instrutor b on a.id_professor = b.id_instrutor
```

```
inner join graduacao.curso c on a.id_curso = c.id_curso;
```

```
select * from relatorio_professores;
```

Link do material de aprofundamento

[Material view](#)

LUIS_DEV



Dúvidas?

LUIS_DEV

